

Discrete Structure

Unit 5. Graph Theory

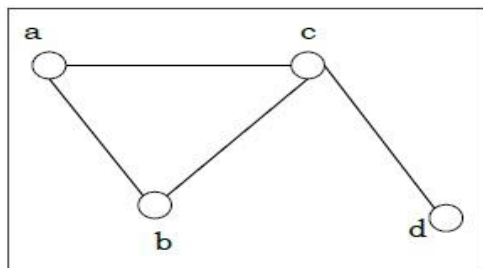
5.1 Graphs:

A graph is a non-linear data structure made up of vertices (nodes) and edges (connections) that represent relationships between objects. Unlike arrays or linked lists, graphs do not store elements in a sequential order.

A graph $G = (V, E)$ consists of V , a nonempty set of vertices (or nodes) and E , a set of edges. Each edge has either one or two vertices associated with it, called its endpoints.

Example:

Let us consider, a Graph is $G=(V,E)$ where $V=\{a,b,c,d\}$ and $E=\{\{a,b\},\{a,c\},\{b,c\},\{c,d\}\}$

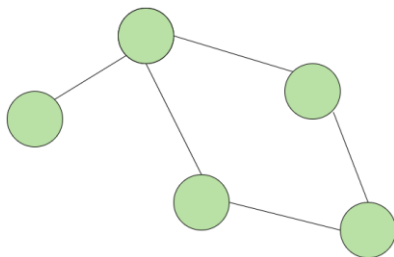


Types of Graphs

There are different types of graphs,

1. Cycle in Graph(in syllabus)

A **cycle** in a graph is a closed path that starts and ends at the same vertex, with all other vertices in the path being distinct. It must contain at least three vertices and no edge is repeated. Cycles are used to identify loops in a graph and distinguish cyclic graphs from acyclic ones.

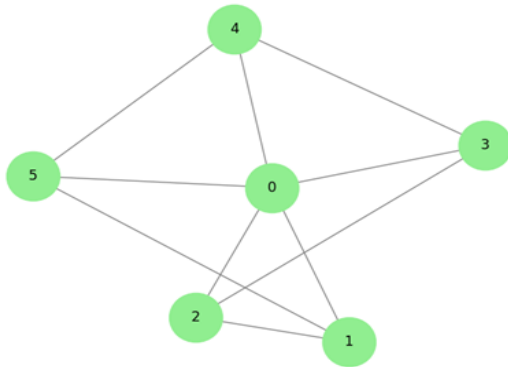


2. Wheel Graph (in syllabus)

A wheel graph is a graph formed by connecting a single central vertex to all vertices of a cycle graph. The central vertex is connected to every other vertex in the cycle.

Example:

The following image shows a wheel graph with six vertices (including the central vertex) –

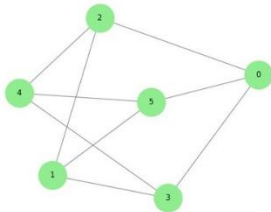


3. Regular Graph(in syllabus)

A regular graph is a type of graph in which each vertex has the same degree, meaning that every vertex is connected to the same number of edges. In other words, all vertices in a regular graph have an equal number of neighbors.

Example: Regular Graph

The following image shows a regular graph –



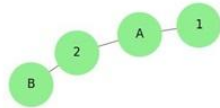
4. Bipartite Graph (in syllabus)

A bipartite graph is a type of graph in which the vertices can be divided into two distinct sets such that no two vertices within the same set are adjacent.

In other words, every edge connects a vertex from one set to a vertex from the other set, and there are no edges connecting vertices within the same set.

Example: Bipartite Graph

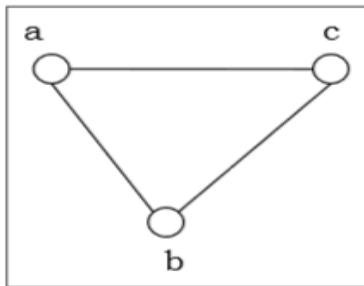
The following image demonstrates a bipartite graph with two sets of vertices –



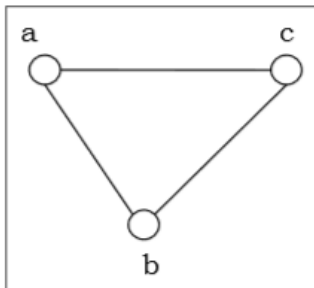
5. **Null Graph** : A null graph has no edges. The null graph of vertices is denoted by

6. **Simple Graph**

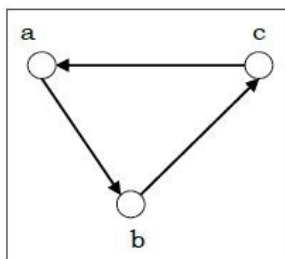
A graph is called simple graph/strict graph if the graph is undirected and does not contain any loops or multiple edges.



7. **Undirected Graph** – Edges have no direction.

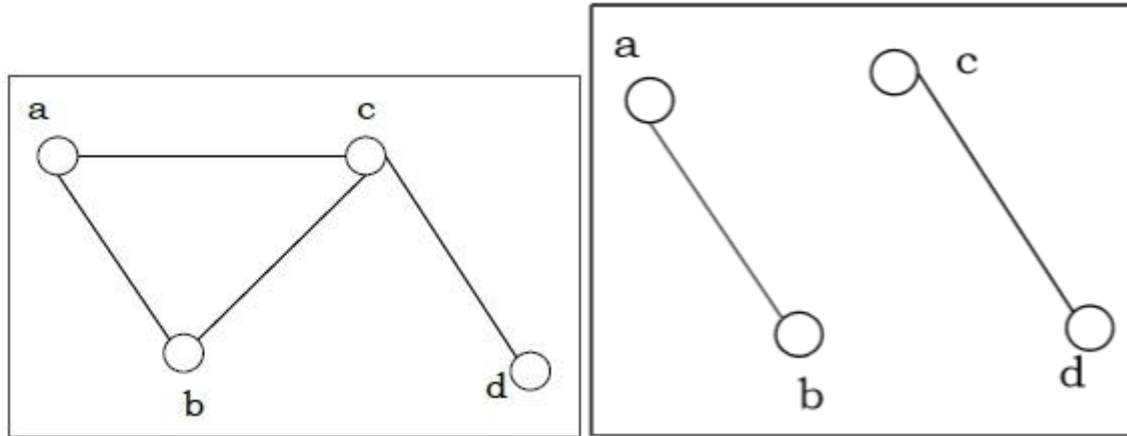


8. **Directed Graph (Digraph)** – Edges have a direction (arrows).



9. Connected and Disconnected Graph

A graph is connected if any two vertices of the graph are connected by a path; while a graph is disconnected if at least two vertices of the graph are not connected by a path



10. Complete Graph(describe below)

Representation of graph

The three common ways to represent a graph are :

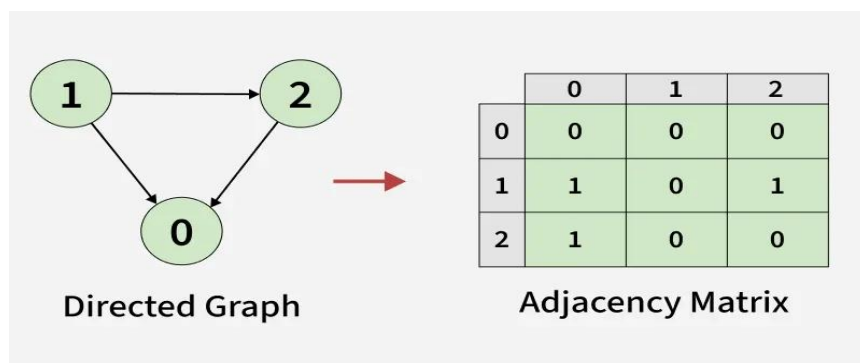
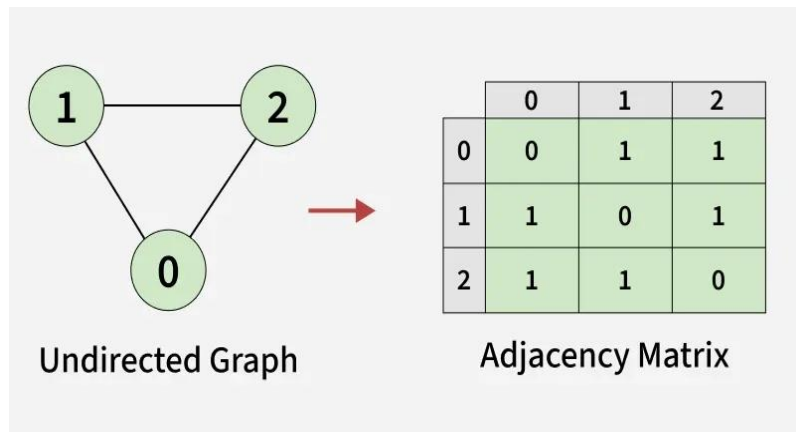
1. **Adjacency Matrix**
2. **Incidence Matrix**
3. **Adjacency List**

Adjacency Matrix

An adjacency matrix is a way of representing a graph as a boolean matrix of (0's and 1's).

Let's assume there are n vertices in the graph So, create a 2D matrix $\text{adjMat}[n][n]$ having dimension $n \times n$.

- If there is an edge from vertex i to j , mark $\text{adjMat}[i][j]$ as 1.
- If there is no edge from vertex i to j , mark $\text{adjMat}[i][j]$ as 0.

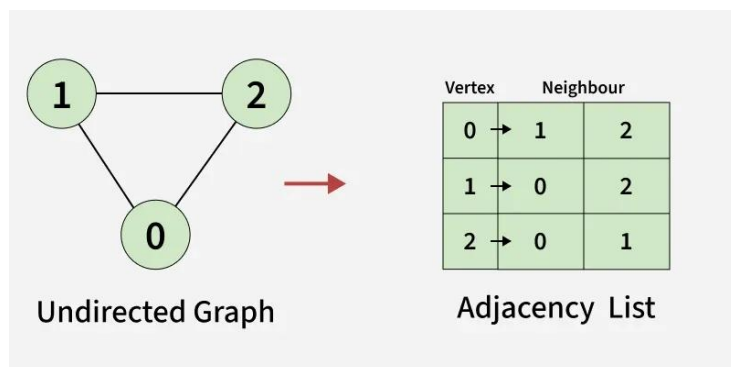


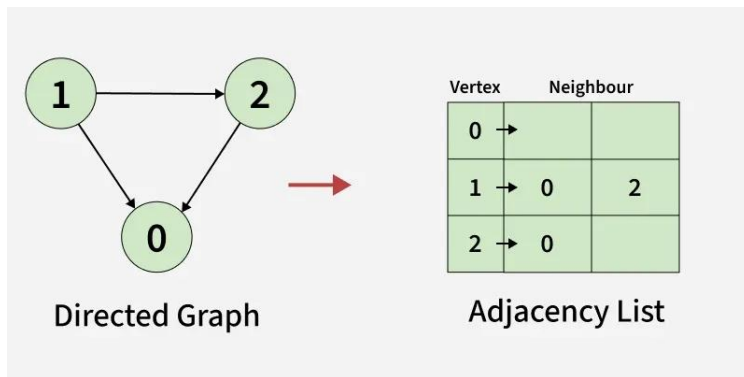
Adjacency List

An array of Lists is used to store edges between two vertices. The size of array is equal to the number of vertices (i.e, n). Each index in this array represents a specific vertex in the graph.

- *adjList[0] will have all the nodes which are connected (neighbour) to vertex 0.*
- *adjList[1] will have all the nodes which are connected (neighbour) to vertex 1 and so on.*

Example





incidence matrix

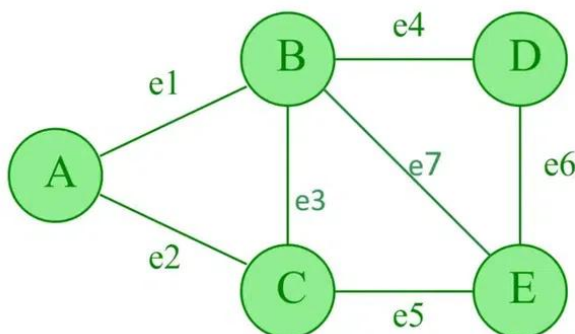
The **incidence matrix** is a way of representing a graph where rows represent vertices and columns represent edges, each entry in the matrix indicates whether a vertex is incident to an edge. Unlike an adjacency matrix, which shows relationships between the vertices of a graph (whether there is an edge between two vertices)

- Rows → Vertices
- Columns → Edges

If a vertex is connected to an edge, we write **1**.

If it is not connected, we write **0**.

Example:

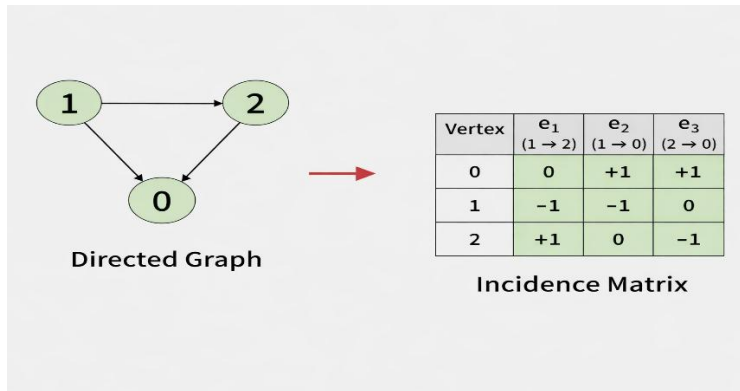


The incidence matrix for the above undirected graph is:

Node/Edges	e1	e2	e3	e4	e5	e6	e7
A	1	1	0	0	0	0	0
B	1	0	1	1	0	0	1

C	0	1	1	0	1	0	0
D	0	0	0	1	0	1	0
E	0	0	0	0	1	1	1

In the case of a directed graph, the incidence matrix will use -1 to represent the direction of the edge. If there is a directed edge from vertex v to vertex u , the row corresponding to v will have a -1, and the row corresponding to u will have a 1.

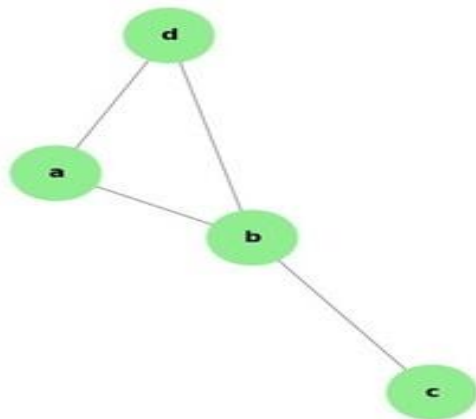


Degree of Vertex

The degree of a vertex in graph theory is the number of edges connected to it. For example, if a vertex has five edges connected to it, its degree is

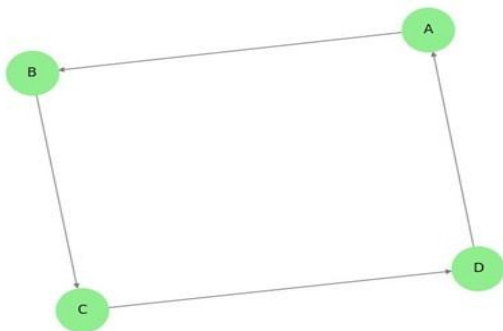
- In an **undirected graph**, the degree of a vertex is the total number of edges connected to it.
- In a **directed graph**, the degree is divided into:
 - **In-degree**: number of incoming edges
 - **Out-degree**: number of outgoing edges

Example 1



- $\deg(a) = 2$, as there are 2 edges meeting at vertex 'a'.
- $\deg(b) = 3$, as there are 3 edges meeting at vertex 'b'.

Example of Degree of Vertex in a Directed Graph



The indegree and outdegree of other vertices are shown in the following table –

Vertex	Indegree	Outdegree
a	1	2
b	2	0
c	2	1
d	1	1
e	1	1
f	1	1
g	0	2

Handshaking Theorem

The Handshaking Theorem states that in any undirected graph, the sum of the degrees of all vertices is equal to twice the number of edges.

$$\sum \deg(v) = 2 |E|$$

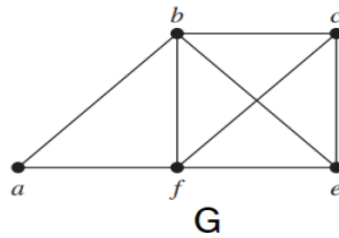


Figure 10: Graph G

Here we have 5 vertices and 8 edges

$\deg(v_i)$ in Graph G	$\sum_{i=1}^m \deg(v_i)$
$\deg(a) = 2$	$\sum_{i=1}^5 \deg(v_i) = \deg(a) + \deg(b) + \deg(c) + \deg(e) + \deg(f)$ $\sum_{i=1}^5 \deg(v_i) = 2 + 4 + 3 + 3 + 4 = 16$ $16 = 2 E $ $ E = 8$
$\deg(b) = 4$	
$\deg(c) = 3$	
$\deg(e) = 3$	
$\deg(f) = 4$	

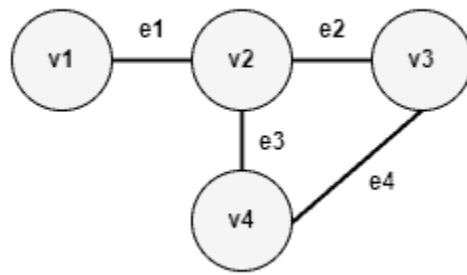
5.2 Connectivity in Graphs:

Connectivity in graphs describes how vertices are linked with each other through edges. It helps us understand whether we can travel from one vertex to another and how strongly the vertices of a graph are related.

Path

A **path** in a graph is a sequence of vertices such that each consecutive pair of vertices is connected by an edge. The length of a path is the number of edges present in it.

PATH EXAMPLES

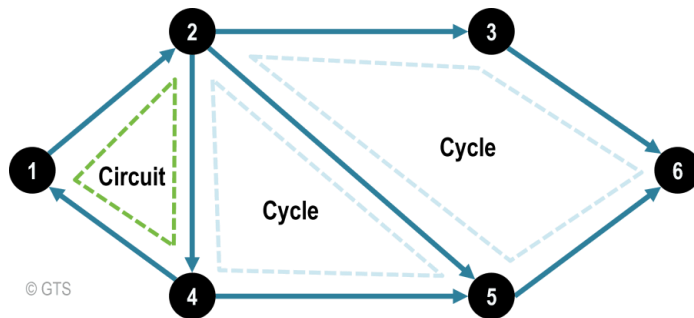


$v1 - (e1) - v2 - (e2) - v3$

$v1 - (e1) - v2 - (e3) - v4 - (e4) - v3$

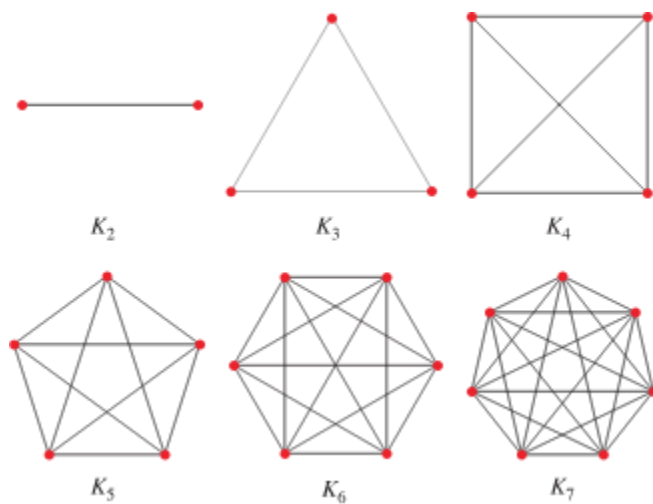
Circuits

A circuit is a sequence of adjacent nodes starting and ending at the same node. Circuits never repeat edges. However, they allow repetitions of nodes in the sequence.



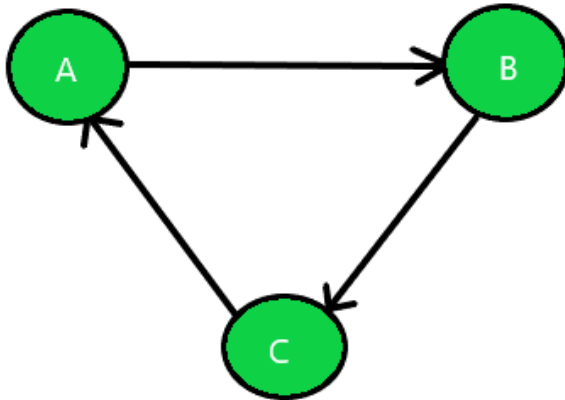
Complete graph

A complete graph is a type of graph in which every pair of distinct vertices is connected by a unique edge. In other words, in a complete graph, every vertex is adjacent to every other vertex.



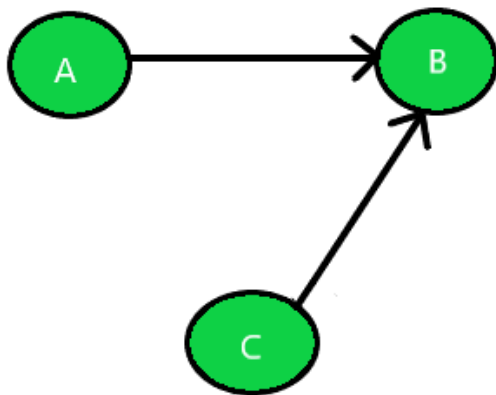
Weakly and Strongly connected graph

A graph is said to be strongly connected if every pair of vertices(u, v) in the graph contains a path between each other. A directed graph is said to be **strongly connected** if for every pair of vertices (u, v) in the graph, there exists a directed path from u to v and a directed path from v to u . This means each vertex is reachable from every other vertex by following the direction of edges.



Path Matrix : A B C
A 1 1 1
B 1 1 1
C 1 1 1

A directed graph is said to be **weakly connected** if the graph becomes connected after ignoring the direction of all edges. In other words, there exists a path between every pair of vertices in the underlying undirected graph.

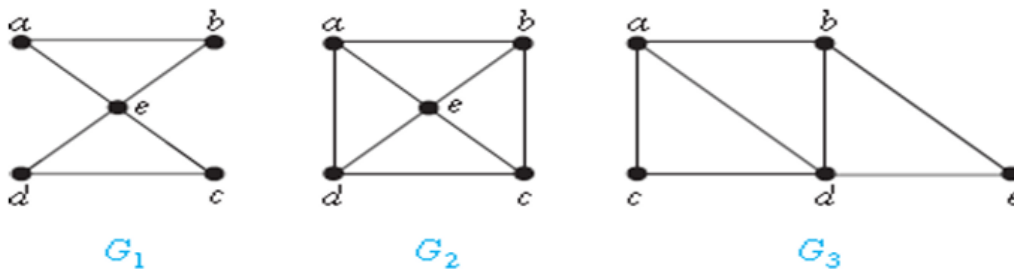


Path Matrix : A B C
A 0 1 0
B 0 0 0
C 0 1 0

Euler Graph

- A graph is called an Euler graph if it contains an Euler circuit.
- An **Euler path** is a path that uses every edge in a graph with no repeats. Being a path, it does not have to return to the starting vertex.
- An **Euler circuit** is a circuit that uses every edge in a graph with no repeats. Being a circuit, it must start and end at the same vertex.
- A connected multigraph with at least two vertices has an Euler circuit if and only if each of its vertices has even degree.

Example: The graph G_1 has an Euler circuit, for example, a, e, c, d, e, b, a . Neither of the graphs G_2 or G_3 have an Euler circuit. G_3 has an Euler path, namely, a, c, d, e, b, d, a, b . G_2 does not have an Euler path.



Conditions for Euler Paths and Circuits

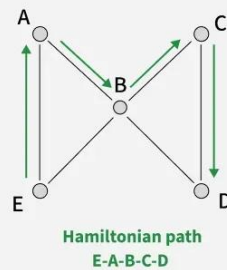
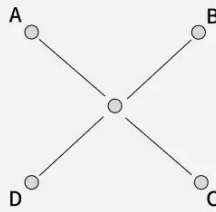
- **Euler Path:** A connected graph has an Euler path if and only if it has exactly zero or two vertices of odd degree.
- **Euler Circuit:** A connected graph has an Euler circuit if and only if every vertex has an even degree.

Hamilton Graph

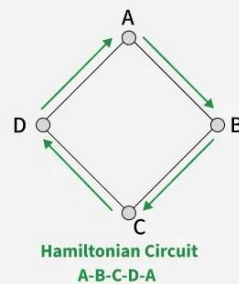
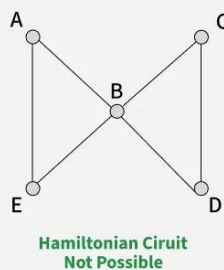
- **Hamiltonian Path:** A path in a graph that visits every vertex exactly once.
- A simple circuit in graph G that passes through every vertex exactly once is called a **Hamiltonian circuit**.
- A **Hamilton graph** is a graph that contains a **Hamilton circuit**.

Example

Hamiltonian Path: A path in a graph that visits every vertex exactly once.



Hamiltonian Circuit: A circuit that passes through every vertex exactly once.



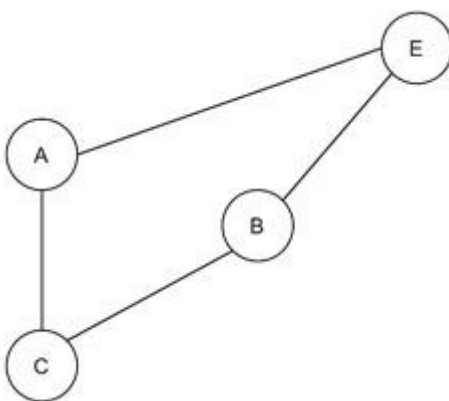
5.3 Planar graph and Planar representation of graph, Graph Coloring:

Planar Graph

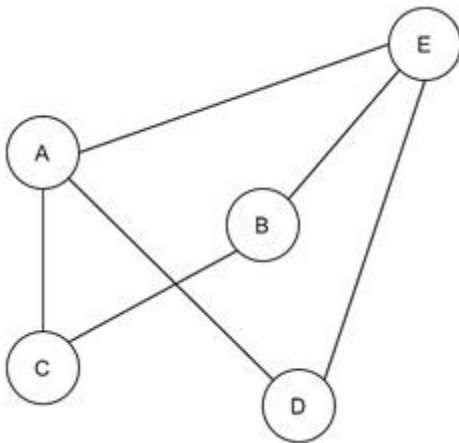
A **planar graph** is a graph that can be drawn on a plane in such a way that **no two edges cross each other**, except at their common endpoints (vertices).

The drawing should be possible without lifting the pen or allowing edges to intersect.

Example

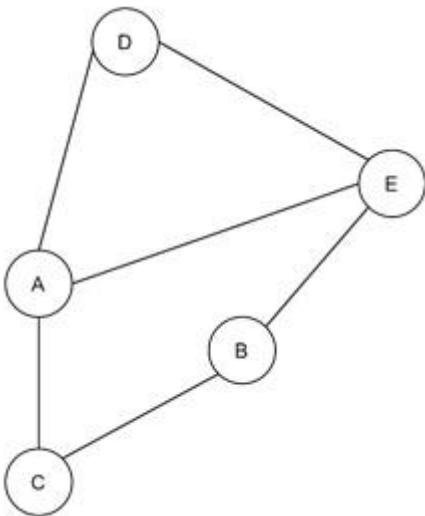


Let us understand this concept with a simple **example**. Take a look at the following graph.



It is a graph with 5 vertices and 6 edges. It is not a planar graph because it has two edges AD and BC which are intersecting each other.

Let's now draw the same graph in a different way; like the one shown below –



Observe that it is the same graph but its vertex D is now placed outside and it has now transformed into a planar graph

These properties are used to determine the planarity of a graph –

- **Face Definition:** A planar graph divides the plane into regions called **faces**. One of these faces is the outer, unbounded region.
- **Euler's Formula:** For any connected planar graph, Euler's formula states that:

$$V - E + F = 2$$

Here, **V** is the number of vertices, **E** is the number of edges, and **F** is the number of faces.

- **Kuratowski's Theorem:**

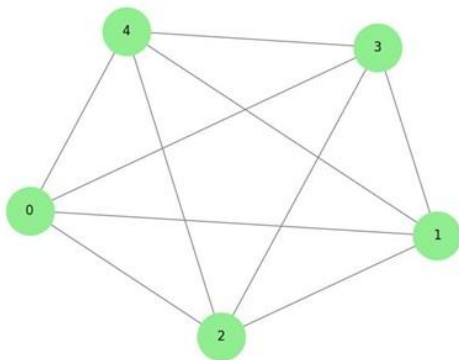
Kuratowski's Theorem provides a condition to determine whether a graph is planar. According to the theorem –

A graph is planar if and only if it does not contain a subgraph that is a subdivision of K_5 (complete graph on 5 vertices) or $K_{3,3}$ (complete bipartite graph on 6 vertices, split into two sets of 3).

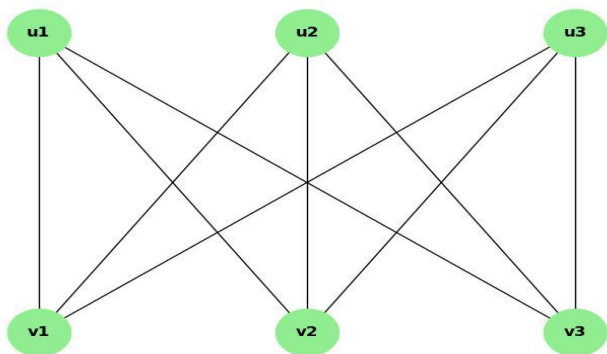
Understanding K_5 and $K_{3,3}$

To be fully familiar with Kuratowski's Theorem, it is important to understand the graphs K_5 and $K_{3,3}$ –

- **K_5 :** It is a complete graph with 5 vertices, where every vertex is connected to every other vertex. It is non-planar because it cannot be drawn without edge crossings.



- **$K_{3,3}$:** A complete bipartite graph with two sets of 3 vertices, where every vertex in one set is connected to all vertices in the other set. It is also non-planar due to edge crossings.



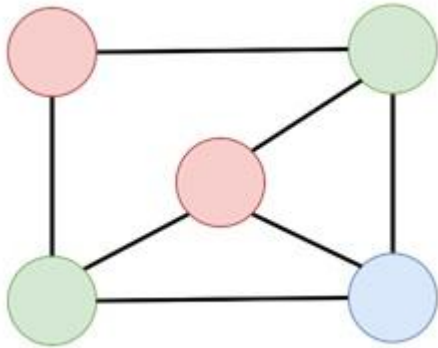
Graph Coloring

Graph coloring is the process of assigning colors to the vertices of a graph such that **no two adjacent vertices have the same color**. The main objective of graph coloring is to use the **minimum number of colors**.

The minimum number of colors required to color a graph is called its **chromatic number**.

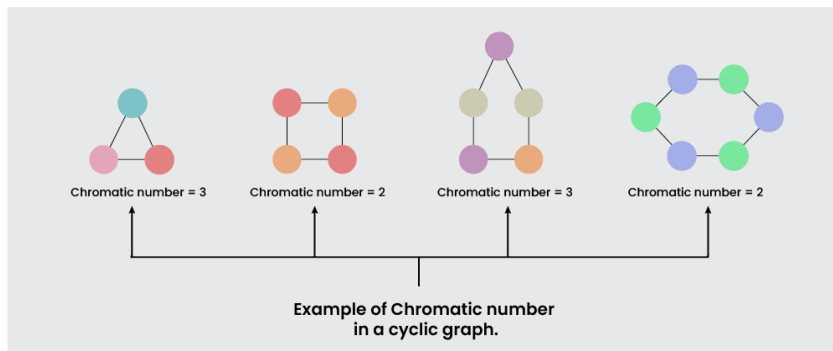
Example:

Take a look at the following graph.



Here, we have 5 vertices and with three colors, we can color the graph. As we need 3 colors as minimum, so the chromatic number for this graph is 3.

Here are some graph and chromatic numbers.



Types of Graph Coloring

- **Vertex Coloring:** Assigning colors to vertices so that no two adjacent vertices have the same color.
- **Edge Coloring:** Assigning colors to edges so that no two edges sharing the same vertex have the same color.
- **Face Coloring:** Assigning colors to faces of a planar graph so that no two faces sharing a boundary have the same color.

5.4 Graph Traversal (BFS and DFS)

Graph traversal is the process of **visiting all vertices (nodes) of a graph** in a systematic way. It is used to **search, explore, and analyze graphs**.

There are two main traversal techniques:

1. **Breadth First Search (BFS)**
2. **Depth First Search (DFS)**

Breadth First Search (BFS)

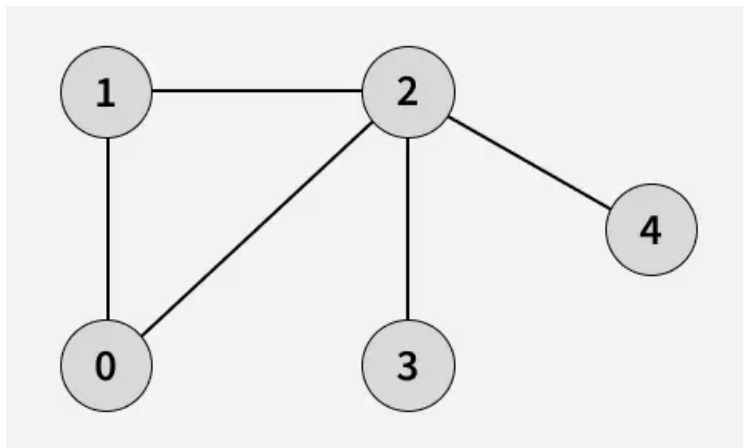
Breadth First Search (BFS) is a graph traversal algorithm that starts from a source node and explores the graph level by level. First, it visits all nodes directly adjacent to the source. Then, it moves on to visit the adjacent nodes of those nodes, and this process continues until all reachable nodes are visited.

It employs the following rules.

- **Rule 1** – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Insert it in a queue.
- **Rule 2** – If no adjacent vertex is found, remove the first vertex from the queue.
- **Rule 3** – Repeat Rule 1 and Rule 2 until the queue is empty.

Example:

Input: $adj[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]$ (**List of Adjacent vertex**)



Output: [0, 1, 2, 3, 4]

Explanation: Starting from 0, the BFS traversal proceeds as follows:

Visit 0 - Print: 0

Visit 1 (neighbor of 0) - Print: 1

Visit 2 (next neighbor of 0) - Print: 2

Visit 3 (first neighbor of 2 that hasn't been visited yet) - Print: 3

Visit 4 (next neighbor of 2) - Print: 4

Depth First Search (DFS)

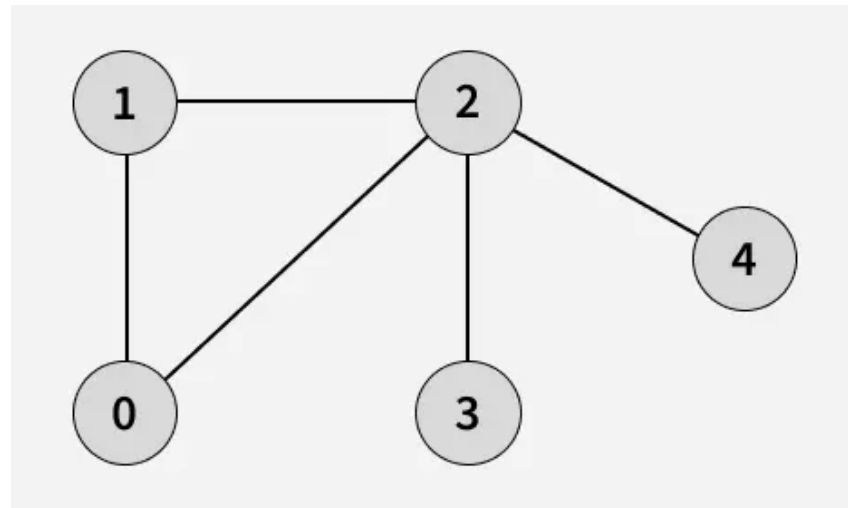
Depth First Search (DFS) is a graph traversal method that starts from a source vertex and explores each path completely before backtracking and exploring other paths. To avoid revisiting nodes in graphs with cycles, a visited array is used to track visited vertices.

It employs the following rules.

- **Rule 1** – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.
- **Rule 2** – If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)
- **Rule 3** – Repeat Rule 1 and Rule 2 until the stack is empty.

Example:

Input: $adj[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]$



Output: $[0, 1, 2, 3, 4]$

Explanation: The source vertex is 0. We visit it first, then we visit its adjacent.

Start at 0: Mark as visited. Print 0.

Move to 1: Mark as visited. Print 1.

Move to 2: Mark as visited. Print 2.

Move to 3: Mark as visited. Print 3.(backtrack to 2)

Move to 4: Mark as visited. Print 4(backtrack to 2, then backtrack to 1, then to 0).

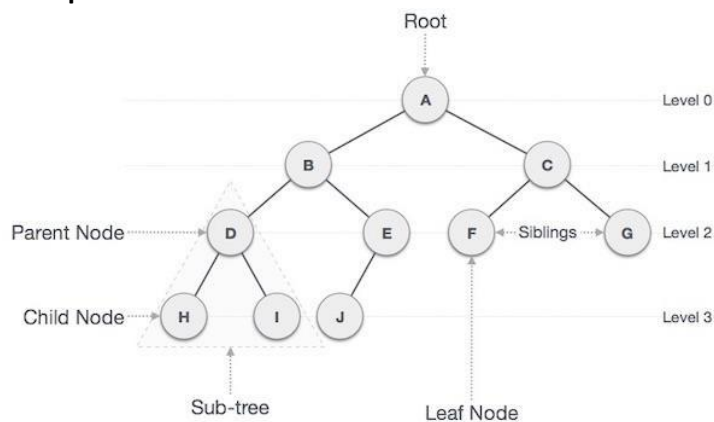
Note that there can be more than one DFS Traversals of a Graph. For example, after 1, we may pick adjacent 2 instead of 0 and get a different DFS.

5.5 Trees:

Introduction and Applications

A tree is a **hierarchical** data structure used to organize and represent data in a **parent–child** relationship. It consists of nodes, where the topmost node is called the **root**, and every other node can have one or more **child nodes**.

Example:



Application of Tree:

- Trees are used to represent hierarchical data like file systems, organization charts, and XML/HTML documents.
- Trees help store, search, insert, and delete data efficiently in databases.
- Trie trees are used in text processing, such as auto-complete, prefix searching, and spell checking.
- Trees are used in computer networks for routing data using spanning trees and shortest path trees.

M-Ary Tree (N-ary Tree)

An M-ary tree is a tree where each node has at most **M** children where each of the children are non-overlapping **M-ary** trees. For example, a 3-ary tree or ternary tree restricts each node to having at most three children. A quaternary tree limits its children to four. Figure 1 shows two examples of a ternary tree and a perfect quaternary tree of height 2.

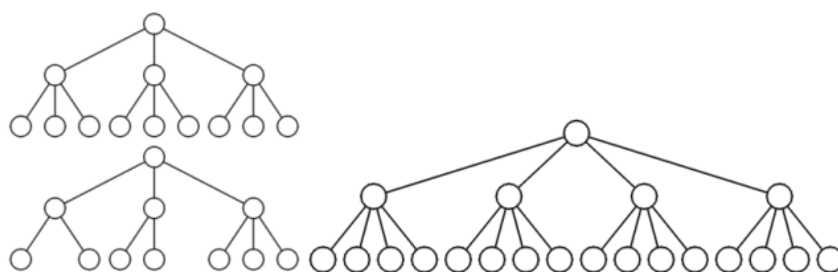
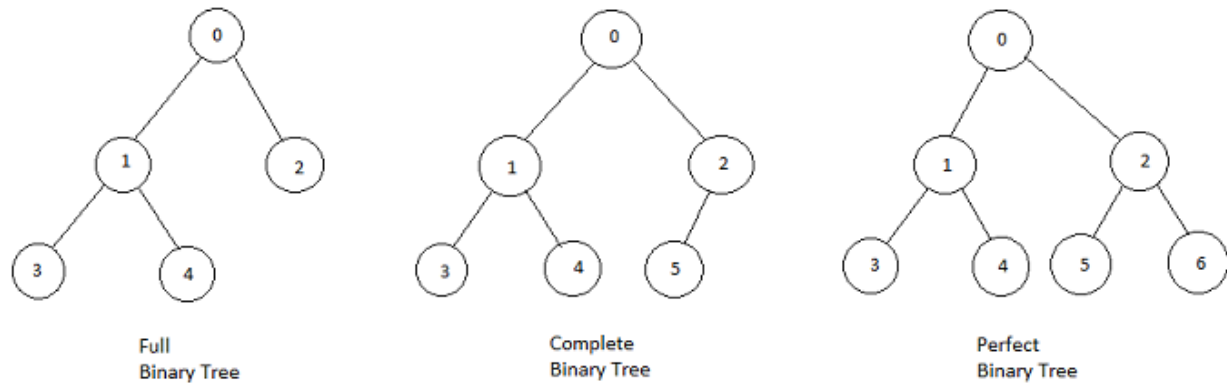


Figure 1. Ternary and a quaternary tree.

Binary Tree

A **Binary Tree Data Structure** is a hierarchical data structure in which each node has at most two children, referred to as the left child and the right child. An M-ary tree will be binary when $M=2$

Examples



Properties of Binary Tree

Here are the properties of Binary Tree

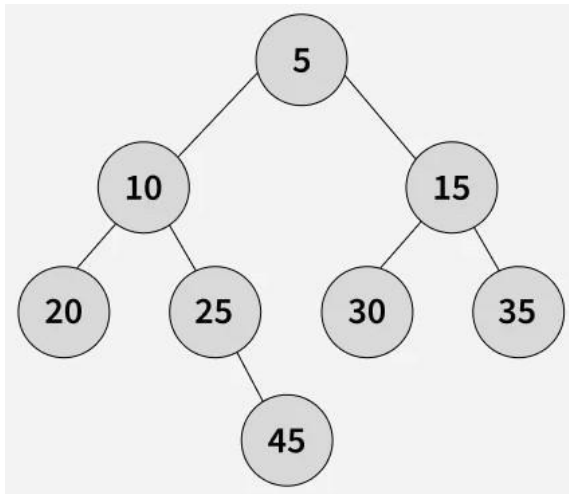
- **Maximum nodes at level l** in a binary tree is 2^{l-1} , where the root is at level 1.
- **Maximum nodes in a binary tree of height n** is $2^n - 1$, where height is the number of nodes from root to the deepest leaf and a single-node tree has height 1.
- **Minimum height (or levels) of a binary tree with n nodes** is $\log_2(n + 1)$.
If the height of a leaf node is taken as 0, then it becomes $\log_2(n + 1) - 1$.
- A binary tree with L leaf nodes has at least $\log_2 L + 1$ levels.
- In a binary tree where each node has **0 or 2 children**, the **number of leaf nodes is always one more than the number of nodes with two children**.

Depth of a tree

The depth of a tree is the maximum number of edges from the root node to any leaf node, representing the longest path in the tree. The root node is at depth 0, with depths increasing by one for each level down. It is equivalent to the maximum height of the tree.

- The **depth** of a node is the number of edges present in path from the root node of a **tree** to that node.
- The **height** of a node is the maximum number of edges from that node to a leaf node in its **subtree**.

Example:



Output: Depth of node 25 = 2

Height of node 25 = 1

Tree Traversals

Traversal is a process to visit all the nodes of a tree and may print their values too. Because, all nodes are connected via edges (links) we always start from the root (head) node.

There are three ways which we use to traverse a tree –

- In-order Traversal
- Pre-order Traversal
- Post-order Traversal

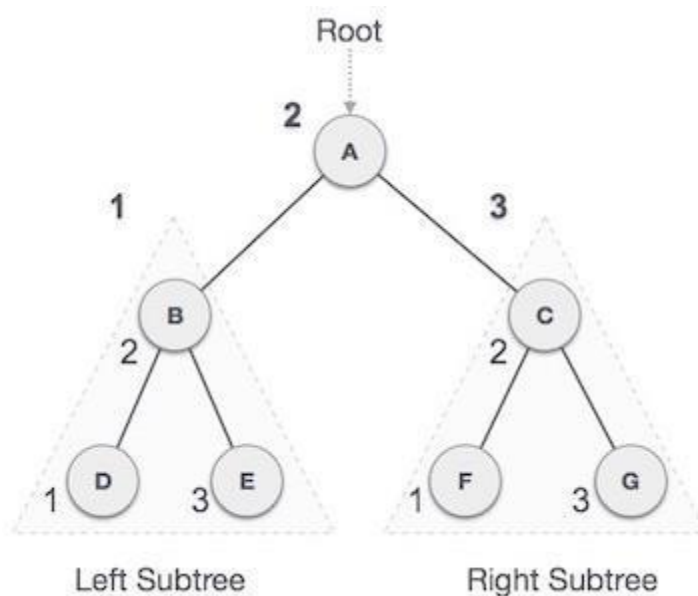
1) In-order traversal

In this traversal method, the left subtree is visited first, then the root and later the right sub-tree. We should always remember that every node may represent a subtree itself.

Algorithm

1. First, visit all the nodes in the left subtree
2. Then the root node
3. Visit all the nodes in the right subtree

Example :



We start from **A**, and following in-order traversal, we move to its left subtree **B**. **B** is also traversed in-order. The process goes on until all the nodes are visited. The output of in-order traversal of this tree will be –

D B E A F C G

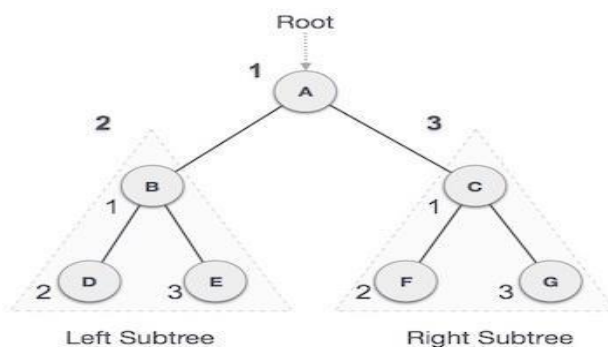
2) Preorder traversal

In this traversal method, the root node is visited first, then the left subtree and finally the right subtree.

Algorithm

1. Visit root node
2. Visit all the nodes in the left subtree
3. Visit all the nodes in the right subtree

Example:



We start with **A**, and following pre-order traversal, we first visit **A** itself and then move to its left subtree **B**. **B** is also traversed pre-order. The process goes on until all the nodes are visited. The output of pre-order traversal of this tree will be –

A → B → D → E → C → F → G

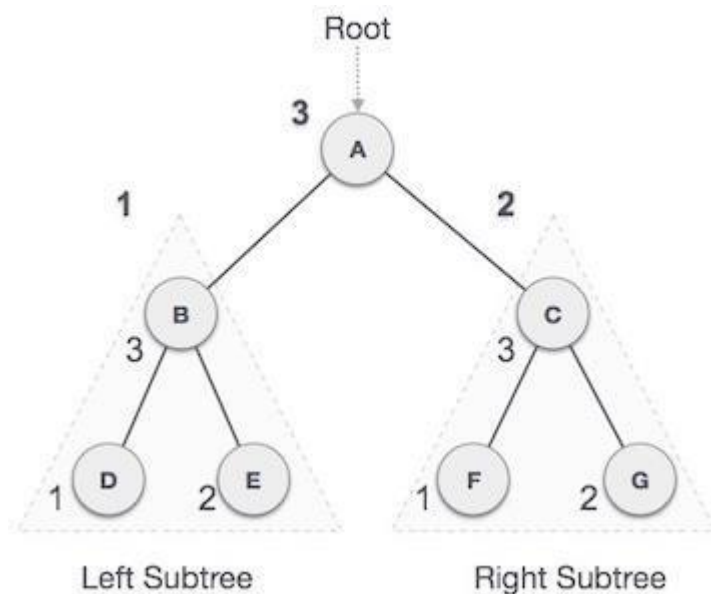
Post-order traversal

In this traversal method, the root node is visited last, hence the name. First we traverse the left subtree, then the right subtree and finally the root node.

Algorithm

1. Visit all the nodes in the left subtree
2. Visit all the nodes in the right subtree
3. Visit the root node

Example:



We start from **A**, and following pre-order traversal, we first visit the left subtree **B**. **B** is also traversed post-order. The process goes on until all the nodes are visited. The output of post-order traversal of this tree will be

D → E → B → F → G → C → A